Proc. of the International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME 2026)
15-17 October 2026, Bali, Indonesia

Characterizing Speculative Decoding Dynamics for Large Language Models on Consumer-Class GPUs: An Empirical Study on RTX 4090

Samsu Sempena
The University of Sydney
ssem0956@uni.sydney.edu.au

Zhuohan Cui
The University of Sydney
zcui0321@uni.sydney.edu.au

***Abstract*—Efficient local LLM inference on consumer-class GPUs is increasingly important for privacy-preserving and real-time intelligent systems, but speculative-decoding behavior on RTX-class mobile hardware remains under-characterized relative to data-center reports. We present a controlled empirical study on one RTX 4090 Laptop using a Qwen2.5-3B-Instruct target with same-family 0.5B/1.5B drafts across GSM8K, a 5-subject MMLU subset, and CNN/DailyMail, with frozen manifests and matched prompts. We evaluate 12 speculative configurations ($k \in \{4, 8, 16\}$, deterministic/stochastic regimes, two draft sizes) plus regime-matched baselines, and report latency, throughput, acceptance dynamics (α, B_{eff}), and task-level quality deltas. The best observed speedup is 0.5B, k=4, deterministic (S=3.0674, 95% CI $[3.029, 3.104]$, $p < 10^{-4}$). A key system-level finding is that speedup decreases monotonically as k increases ($\bar{S}_{k=4} = 2.9164$, $\bar{S}_{k=8} = 2.4029$, $\bar{S}_{k=16} = 1.8019$) even while B_{eff} rises, indicating a verification-and-drafting overhead inflection where larger blocks no longer improve end-to-end efficiency on this hardware profile. Deterministic decoding is faster in 5 of 6 matched draft–k pairs (mean $\Delta S_{\text{det-stoch}} = 0.0568$); quality drift is modest on MMLU/CNN-DailyMail but more variable on GSM8K under stochastic decoding. We conclude with deployment-oriented design guidance for choosing draft size and k on consumer GPUs, and derive an adaptive drafting policy discussion from these fixed-policy observations.**

***Index Terms*—speculative decoding, large language models, inference acceleration, benchmarking, reproducibility**

I. Introduction

Autoregressive decoding in large language models (LLMs) is inherently sequential: each generated token requires a full forward pass of the target model. This constraint directly limits its single-stream throughput in interactive inference settings. Speculative decoding mitigates this bottleneck by adding a smaller draft model that proposes a length-k token block, followed by one target-side verification step; modified rejection sampling preserves the target distribution [1], [2].

Although speculative decoding is now well studied, deployment guidance remains uncertain for consumer-GPU settings. Reported gains in the literature are often obtained on data-center hardware and are sensitive to model pairing, proposal length, and sampling regime. For practitioners operating on a single RTX-class GPU, the key questions are practical: which draft size is worthwhile, which k is optimal, and whether deterministic versus stochastic decoding changes the speed–quality frontier.

This direction is increasingly important because real-world LLM usage is moving toward consumer-class deployment constraints: local assistants, student and developer laptops, and privacy-sensitive workloads where cloud offload is undesirable or expensive. In this setting, efficiency is not only a cost optimization problem; it is an accessibility and feasibility requirement. Inference methods that are robust on consumer hardware can widen practical LLM adoption beyond data-center environments.

Beyond pure NLP serving, this deployment constraint is also relevant to mechatronic and cyber-physical workflows that require low-latency local language interaction, including edge-side instruction parsing, operator assistance, and offline diagnostics. We therefore position this paper as a hardware-aware characterization study for local intelligent systems, not as a new decoding algorithm.

A. Research questions

We address three research questions:

RQ1. How much end-to-end speedup does vanilla speculative decoding deliver against autoregressive decoding on a single RTX 4090 Laptop with a 3 B target, and how close is this to a practical hardware-constrained upper bound?

RQ2. How do draft model size and draft length jointly determine token acceptance and net latency, and what do these trends imply about memory-traffic and kernel-launch overhead on consumer GPUs?

RQ3. How sensitive is the speed–quality trade-off to the sampling regime (deterministic vs. stochastic) and to task entropy?

B. Contributions

This paper makes the following contributions:

1) A reproducible consumer-GPU protocol with frozen sample manifests, matched prompts, and two de-

coding regimes across GSM8K, MMLU subset, and CNN/DailyMail.
2) A complete 12-configuration speculative grid (two draft sizes, $k \in \{4, 8, 16\}$, deterministic/stochastic) with regime-matched baselines and per-configuration reporting of latency, throughput, acceptance, and quality deltas.
3) Empirical evidence that, in the current RTX4090 CUDA-graph run family, speedup decreases with larger k (highest at k=4), with best observed performance at 0.5B, k=4, deterministic (S=3.0674), and a deterministic advantage in 5 of 6 matched draft–k pairs.
4) A statistical reporting protocol for camera-ready evaluation: paired bootstrap confidence intervals, one-sided significance tests for $S>1$, and corrected pairwise comparisons for winner selection.
5) A practical operating recommendation and a transparent statement of current reproducibility limitations (artifact consistency across reruns), together with adaptive-policy design guidance derived from these fixed-policy observations.

The rest of the paper is organised as follows. Section II reviews related work. Section III defines the experimental protocol; Section IV defines metrics. Section V describes implementation and reproducibility controls. Section VI reports the main empirical findings. Section VII outlines an adaptive speculative-strategy discussion derived from the observed k-efficiency inflection. Section VIII discusses implications, limitations, and conclusion.

II. Related Work

a) Vanilla speculative decoding.: Leviathan *et al.* [1] introduced the canonical draft–verify scheme: a smaller *draft* model proposes k tokens autoregressively and the larger *target* verifies them in a single parallel forward pass, with modified rejection sampling guaranteeing that the output distribution is identical to that of the target. Chen *et al.* [2] concurrently proposed an equivalent scheme. The expected per-cycle latency is

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \qquad T_{\text{draft}} = k \cdot t_{\text{draft step}}, \qquad (1)$$

where $\tau \in [1, k+1]$ is the expected number of accepted tokens per cycle. Wall-clock speedup $S = L_{\text{target}}/L$ therefore depends on the draft/target cost ratio and the empirical token acceptance rate α. Stern *et al.* [3] earlier explored blockwise parallel decoding within a single model.

b) Reducing drafting cost.: Medusa [4] eliminates the external draft by attaching multiple prediction heads to the target model and verifying the resulting candidate tree in one forward pass. The EAGLE family [5]–[7] keeps an external draft but conditions it on the target's hidden features, reporting substantial speedup under their respective settings. Despite these refinements, T_{draft} remains linear in k, which forces EAGLE-3 to a single transformer layer and caps achievable speedups at $\sim$2–3$\times$ on strong GPUs.

c) Diffusion-based drafters.: Recent work attacks the linear-in-k drafting bottleneck directly. DiffuSpec [8] and SpecDiff-2 [9] explore diffusion-based parallel drafting to improve speculative-decoding throughput. PARD [10] instead focuses on low-cost parallel draft model adaptation for accelerating LLM inference. DFlash [11] closes this gap with a lightweight (5-layer) *block-diffusion* drafter conditioned on target hidden features, reporting up to $5\times$ end-to-end speedup over autoregressive baselines on H200/B200 hardware.

d) Precision and systems deployment.: Deployment-oriented inference work shows that quantization and memory traffic are central to practical throughput gains on constrained hardware. Low-precision techniques such as LLM.int8() [12] reduce weight movement and can interact with speculative decoding in two opposing ways: lower per-step cost for draft/verify passes, but possible shifts in acceptance behavior when verifier and drafter precision settings differ. This interaction motivates reporting speed and quality jointly rather than throughput alone.

e) Distributed and edge relevance.: In distributed and mobile computing settings, local inference can reduce round-trip latency and cloud dependency for interactive workloads. Within this context, speculative decoding is relevant not only to chatbot serving but also to edge-side language interfaces in intelligent devices, where bounded latency and predictable runtime behavior are as important as peak throughput.

f) Position of this work.: This work focuses on a controlled consumer-GPU evaluation of vanilla speculative decoding with same-family model pairing (Qwen2.5-3B target, 0.5B/1.5B drafts) under deterministic and stochastic regimes. Rather than competing with data-center throughput claims, we characterise practical operating points on a single RTX 4090 Laptop, including acceptance dynamics, quality drift, and runtime-sensitive configuration choices. We then use these results to motivate a future adaptive cascaded speculative decoding (ACSD) extension aimed at reducing long-tail latency under the same hardware budget.

III. Experimental Protocol

A. Datasets and sample sizes

We evaluate on three benchmarks spanning reasoning, knowledge, and summarization:

1) **GSM8K** [13] test subset: 300 samples.
2) **MMLU** [14] subset: 500 samples (5 subjects $\times$ 100 each).
3) **CNN/DailyMail** [15] subset: 200 samples.

Sampling policy: Fixed random seed $= 42$, stratified sampling where applicable, and a frozen sample-ID manifest generated before experimentation. Each output CSV records seed and sample identifier, enabling paired analysis at sample level.

B. Prompt set

We use the following fixed prompt templates for each task:

GSM8K:

TABLE I
HARDWARE PROFILE USED FOR THE RTX4090 RUN FAMILY.

Component	Specification
GPU	NVIDIA RTX 4090 Laptop GPU
GPU memory	24 GB GDDR6
Memory bandwidth	576 GB/s
CUDA cores	9728
Architecture	Ada Lovelace
Power profile	Laptop TGP class (80–150 W range)
Software stack	PyTorch + Transformers (pinned env)

You are a careful math assistant. Solve the problem step by step and end with Final Answer: <number>.
Question: {question}

MMLU:

You are a knowledgeable assistant. Choose exactly one option.
Question: {question}
Options: A. {A} B. {B} C. {C} D. {D}
Answer:

CNN/DailyMail:

Summarize the following article in 3–4 sentences, preserving key facts and entities.
Article: {article}
Summary:

a) Objective and hypotheses.: The study quantifies practical speculative-decoding gains on consumer hardware under fixed data and prompt controls. We test four hypotheses:

- **H1:** Same-family speculative decoding yields positive net speedup ($S > 1$) over autoregressive decoding.
- **H2:** Increasing draft length from k=4 to k=8 improves speedup, while k=16 may reduce gains due to additional drafting and verification overhead.
- **H3:** A larger draft (1.5B) achieves higher acceptance and higher net speed than a smaller draft (0.5B).
- **H4:** Deterministic decoding is more runtime-efficient and quality stable than stochastic decoding for the same draft and k.

b) Hardware and software controls.: All experiments run on one NVIDIA RTX 4090 Laptop (24GB VRAM) with a fixed software stack (PyTorch, Transformers, and tokenizer settings pinned by the project environment). We use the same runtime entry points and prompt templates for all configurations, and execute runs serially to avoid cross-run contention.

c) CUDA-graph runtime setup.: To reduce per-step launch overhead, we use a CUDA-graph-capable verify path. For each k, we keep fixed step shapes, pre-allocate static draft/verify buffers, and warm up before optional `torch.cuda.CUDAGraph` capture. During decoding, each cycle updates buffer contents and replays the graph when shape guards hold; otherwise it falls back to eager execution. This preserves output semantics while reducing host-side overhead in short speculative cycles.

d) Model pairing.: Target model: Qwen2.5-3B-Instruct. Draft models: Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct. All models are same-family to minimise tokenizer mismatch and style drift.

e) Task suite and manifests.: We evaluate on fixed manifests reused across all runs:

- GSM8K arithmetic QA (300 samples), treated as logical reasoning with strict step consistency;
- MMLU subset with 5 subjects (500 samples), treated as knowledge retrieval from a broad expert prior;
- CNN/DailyMail summarisation (200 samples), treated as long-context compression akin to operational log summarisation.

Each configuration therefore processes 1,000 prompts.

f) Decoding regimes and grid.: Two regimes are evaluated:

- **Deterministic:** greedy decoding (T=0.0, top-p=1.0; sampling disabled).
- **Stochastic:** sampling with T=0.7, top-p=0.9, and no top-k truncation.

For each draft model, we sweep $k \in \{4, 8, 16\}$, producing 12 speculative configurations. We also run one autoregressive baseline per regime for anchored latency and throughput reporting.

g) Endpoints.: The primary endpoint is paired wall-clock speedup $S = L_{\text{AR}}/L_{\text{spec}}$ as reported by the experiment summary artifact for each draft–k–regime combination. Secondary endpoints include acceptance rate α, effective accepted block size B_{eff}, mean per-sample latency, throughput (tokens/s), and task-level quality deltas. Camera-ready statistical reporting uses paired bootstrap confidence intervals (10,000 resamples) and one-sided tests for S>1, with corrected pairwise winner tests.

IV. EVALUATION METRICS

Table II defines the exact reporting metrics used in this study.

A. Success criteria

Primary:

1) Mean speedup $S \geq 1.3\times$ with CI_S lower bound > 1.0.
2) Absolute quality drop $\leq$ 1.0 point on GSM8K and MMLU in deterministic regime.

Secondary: Identify best (draft size, k) maximizing S while satisfying quality constraints, and report pairwise significance for winner selection with multiple-testing correction.

V. IMPLEMENTATION AND REPRODUCIBILITY

a) Code organisation.: The experiment pipeline is implemented under the project source directory with modular components for configuration, data loading, autoregressive baselines, speculative decoding, metric aggregation, and runtime orchestration. The notebook driver executes these modules in fixed phase order and writes artifacts to the results directory.

b) Model loading and precision.: In the current configuration, target and draft models are loaded in FP16 for the main sweep, with quantisation controls retained in configuration for alternative runs. Tokenizer and generation limits are held constant across baselines and speculative runs to preserve paired comparability.

TABLE II
EXACT METRIC DEFINITIONS FOR REPORTING.

Metric	Symbol	Computation	Unit	Direction
End-to-end latency	T	completion_time − request_start	s	Lower
Throughput	R_{tok}	output_tokens / T	tokens/s	Higher
Time-to-first-token	TTFT	first_token_time − request_start	ms	Lower
Time-per-output-token	TPOT	(completion_time − first_token_time) / output_tokens	ms/token	Lower
Acceptance rate	α	accepted_draft_tokens / proposed_draft_tokens	ratio	Higher
Effective block size	B_{eff}	accepted_draft_tokens / verify_steps	tokens/step	Higher
Relative speedup	S	baseline_latency / speculative_latency	×	Higher
GSM8K quality	Q_{gsm}	correct / total	%	Higher
MMLU quality	Q_{mmlu}	correct / total	%	Higher
CNN/DailyMail quality	Q_{sum}	standard ROUGE-L F1	score	Higher
Quality delta	ΔQ	$Q_{\text{spec}} - Q_{\text{base}}$	points	≈ 0 or $+$
Speedup confidence interval	CI_S	paired bootstrap percentile CI of S (10k resamples)	×	Higher lower-bound
Speedup significance	p_S	one-sided paired test for $H_0 : S \leq 1$	p-value	Lower
Speedup stability	σ_S	std(S over seeds)	×	Lower

c) Baselines.: We run one autoregressive baseline per regime (deterministic and stochastic) with the same manifests and output-length constraints as speculative runs. Each baseline artifact stores per-sample latency, output length, and task predictions.

d) Speculative decoding implementation.: The speculative loop follows standard token-level draft–verify logic. At each cycle, the draft proposes up to k tokens autoregressively; the target verifies the proposal in one pass over the extended prefix; the accepted prefix is committed up to the first mismatch; and one correction token is added from the target distribution when needed. This procedure preserves target-level generation semantics while exposing acceptance statistics needed for analysis. Figure 1 summarizes this implementation flow.

e) CUDA-graph overhead model and observed effect.: For a speculative cycle, we model runtime as

$$T_{\text{cycle}} = k\, t_{\text{draft}} + t_{\text{verify}} + n_{\text{launch}}\, t_{\text{launch}}, \tag{2}$$

where $n_{\text{launch}}\, t_{\text{launch}}$ captures host-side kernel-launch overhead. With successful graph replay, launch cost is reduced to a lower replay overhead term:

$$T^{\text{graph}}_{\text{cycle}} \approx k\, t_{\text{draft}} + t_{\text{verify}} + n_{\text{replay}}\, t_{\text{replay}}, \qquad t_{\text{replay}} \ll t_{\text{launch}}. \tag{3}$$

In the present RTX4090 run family, capture-rate metadata is 0, so the main results reflect fallback execution rather than sustained graph replay. A separate paired GPU-optimization toggle microbenchmark (12 matched samples, k=8) reports mean TPOT 32.84 ms/token (off) versus 33.30 ms/token (on), with mean delta $+0.46$ ms/token (95% CI $[0.14, 0.83]$), indicating no TPOT benefit under the current capture conditions.

f) Regimes and outputs.: The same verification core is used for deterministic and stochastic modes, with only sampling policy changed by regime parameters. Each configuration writes a dedicated CSV artifact with per-sample runtime, token counts, and quality-relevant outputs.

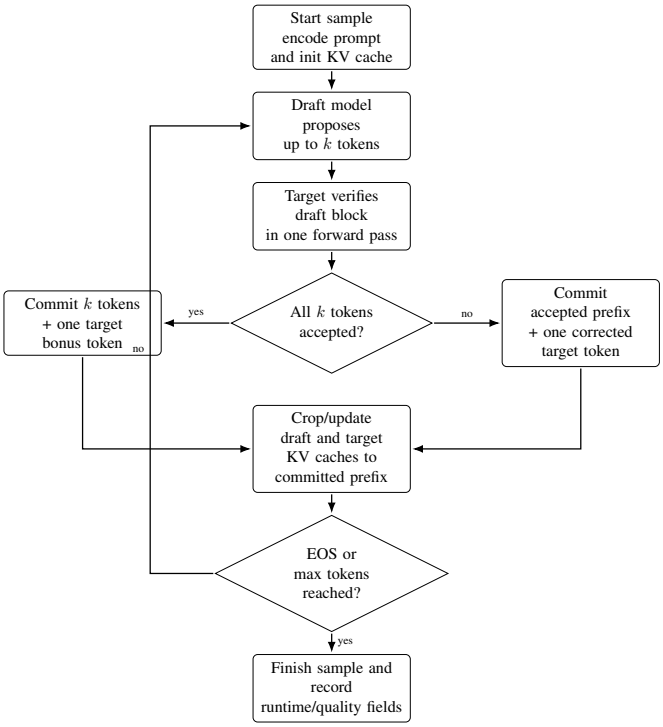

Fig. 1. Vanilla speculative decoding implementation flow used in Phases 3–4. The target model remains the verifier of record; accepted draft prefixes are committed and caches are cropped to preserve exact target-consistent decoding.

VI. RESULTS

This section reports fixed-policy speculative-decoding outcomes from the RTX4090 CUDA-graph run family and regime-matched deterministic/stochastic baselines. Headline values are summarized in *all_configs_summary.csv*. We emphasize paired comparative metrics (S, α, B_{eff}) and task deltas to evaluate the speed–quality tradeoff.

Figure 2 complements Table III with a visual baseline-relative view across all configurations, including speedup, latency/throughput deltas, token-timing deltas, and acceptance

TABLE III
RTX4090 CUDA-GRAPH RUN FAMILY SUMMARY (QWEN2.5-3B TARGET). S IS RATIO-OF-MEANS PAIRED SPEEDUP AGAINST REGIME-MATCHED BASELINES. CI_S IS THE PAIRED BOOTSTRAP 95% INTERVAL (10K RESAMPLES OVER MATCHED SAMPLES); p_S IS ONE-SIDED SIGNIFICANCE FOR $H_0: S \leq 1$. α IS TOKEN-LEVEL ACCEPTANCE; B_{EFF} IS ACCEPTED TOKENS PER VERIFICATION CYCLE.

Config	S	CI_S	p_S	α	B_{eff}	ΔGSM8K	ΔMMLU	ΔCNN/DM
baseline, det	1.0000	–	–	–	–	–	–	–
baseline, stoch	1.0000	–	–	–	–	–	–	–
0.5B, k=4, det	**3.0674**	[3.029, 3.104]	$< 10^{-4}$	0.4212	1.66	0.33	0.00	-0.02
0.5B, k=4, stoch	2.9394	[2.900, 2.979]	$< 10^{-4}$	0.4080	1.61	1.67	-0.20	-0.03
0.5B, k=8, det	2.4542	[2.408, 2.500]	$< 10^{-4}$	0.3515	2.72	0.33	0.00	-0.05
0.5B, k=8, stoch	2.4063	[2.359, 2.453]	$< 10^{-4}$	0.3444	2.67	1.00	0.00	-0.38
0.5B, k=16, det	1.8050	[1.751, 1.859]	$< 10^{-4}$	0.2539	3.76	0.33	0.00	-0.01
0.5B, k=16, stoch	1.8185	[1.764, 1.873]	$< 10^{-4}$	0.2516	3.73	0.34	-0.20	-0.19
1.5B, k=4, det	2.8726	[2.839, 2.905]	$< 10^{-4}$	0.4581	1.81	-0.33	0.00	-0.03
1.5B, k=4, stoch	2.7860	[2.754, 2.819]	$< 10^{-4}$	0.4421	1.75	1.67	0.00	-0.34
1.5B, k=8, det	2.4066	[2.366, 2.447]	$< 10^{-4}$	0.3929	3.04	0.00	0.00	-0.06
1.5B, k=8, stoch	2.3444	[2.304, 2.385]	$< 10^{-4}$	0.3822	2.96	3.67	0.20	-0.47
1.5B, k=16, det	1.8068	[1.761, 1.853]	$< 10^{-4}$	0.2968	4.39	0.67	0.00	-0.03
1.5B, k=16, stoch	1.7773	[1.732, 1.825]	$< 10^{-4}$	0.2928	4.36	1.00	-0.20	-0.36

dynamics.

A. RQ1: Does speculative decoding accelerate inference?

All 12 speculative configurations exceed S=1 in the RTX4090 CUDA-graph run family, with observed speedups from 1.7773 to 3.0674. The best configuration is 0.5B with k=4 in deterministic mode. Baseline context from regime-matched baseline files shows mean latencies of 11.41s (deterministic) and 11.65s (stochastic) per sample, so the measured speculative gains are substantial in wall-clock terms for fixed workload and prompts. Paired bootstrap intervals and one-sided tests confirm that all configurations are significantly faster than autoregressive decoding ($p_S < 10^{-4}$ throughout, Table III).

B. RQ2: How do draft length and draft size affect acceptance and speed?

Draft length shows a monotonic decline in speedup over this grid. Averaging over drafts and regimes, mean speedup is highest at k=4 (2.9164), then k=8 (2.4029), then k=16 (1.8019). This is consistent with acceptance dynamics: as k increases, B_{eff} increases (1.7075 → 2.8475 → 4.0600) but α decreases (0.4324 → 0.3678 → 0.2738), and the extra draft/verify cost dominates net benefit.

Draft size has a smaller global effect than k in this grid, and the 0.5B draft is slightly better on mean speedup (mean S: 2.4151 for 0.5B versus 2.3323 for 1.5B).

Table IV summarizes the deterministic endpoints of this non-linear trend and highlights the systems-level interpretation: higher B_{eff} does not guarantee higher S once per-cycle drafting and verification overhead dominate.

C. RQ3: Deterministic versus stochastic regime

Deterministic decoding is faster in 5 of 6 matched draft–k pairs. The deterministic advantage ranges from -0.0135 to 0.1280 in absolute S and averages 0.0568 across the six matched pairs; the single exception is 0.5B, k=16, where stochastic is slightly faster. This still supports a deployment preference for deterministic regime when latency is primary and output diversity is not required.

TABLE IV
DETERMINISTIC BOTTLENECK ANALYSIS AT LOW VS. HIGH k.

Config	α	B_{eff}	S	**Interpretation**
0.5B, k=4	0.4212	1.66	3.0674	Sweet spot
0.5B, k=16	0.2539	3.76	1.8050	Drafting overload
1.5B, k=4	0.4581	1.81	2.8726	Memory pressure
1.5B, k=16	0.2968	4.39	1.8068	Diminishing returns

Quality stability differs by task. CNN/DailyMail remains close to baseline but consistently negative (-0.47 to -0.01). MMLU shifts are small (-0.2 to 0.2). GSM8K shows the largest spread (-0.33 to 3.67), especially under stochastic decoding. Under a conservative quality filter ($\max|\Delta| \leq 1$ across tasks), 9 configurations remain, with best speedup at 0.5B, k=4, deterministic.

The resulting design guideline is clear: when selecting k on consumer GPUs, prioritize lower per-cycle runtime over maximal accepted-block growth.

Implementation details for the CUDA-graph-capable runtime setup are described in Section III.

D. Summary of empirical operating point

The current evidence supports a practical recommendation: use deterministic speculative decoding with a smaller block (k=4), where speedup is highest, and prefer the 0.5B draft for this hardware profile. Higher k increases accepted block size but lowers net speedup in this setup, indicating that acceptance gains no longer offset added cycle cost.

VII. DISCUSSION ON ADAPTIVE SPECULATIVE STRATEGIES DERIVED FROM EMPIRICAL FINDINGS

The current empirical study identifies the strongest fixed-policy speedup at smaller block size (k=4), with net speed degrading as k increases even when accepted block length

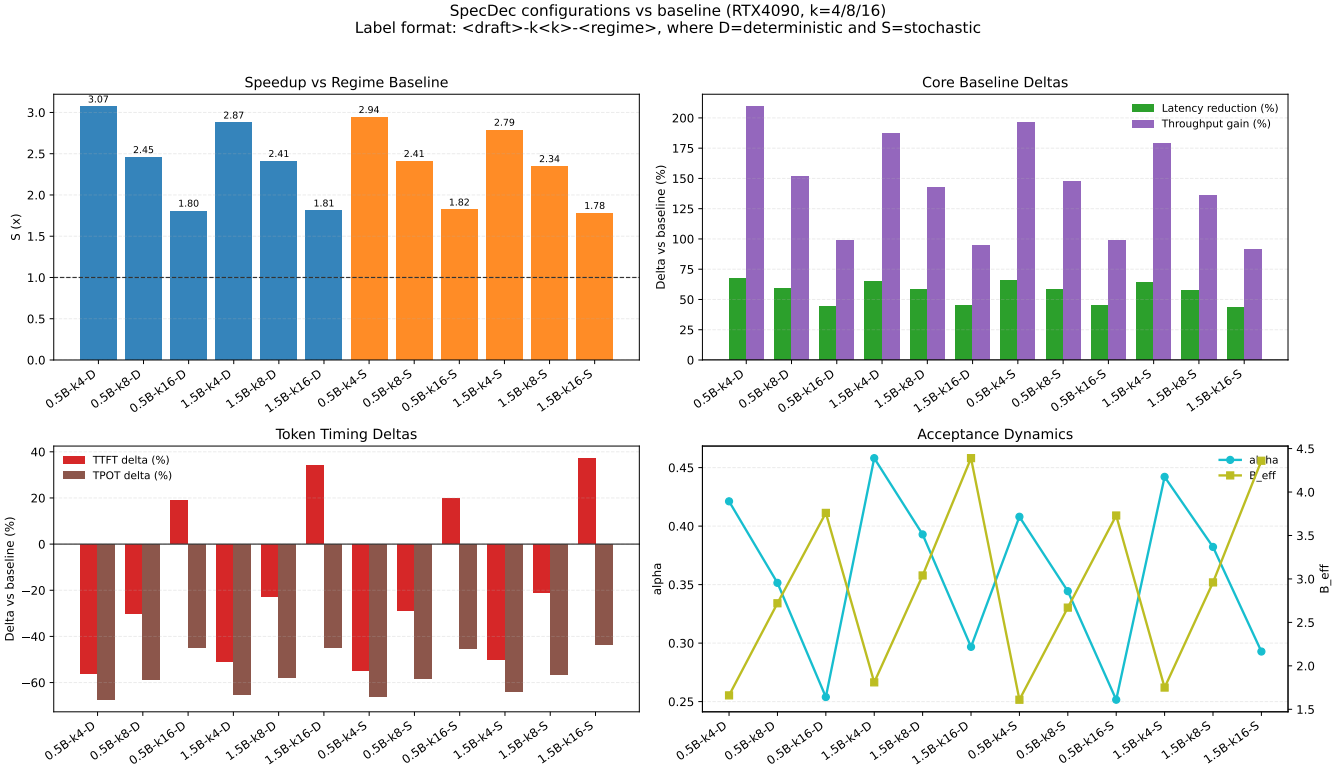

Fig. 2. Speculative-decoding metrics for all RTX4090 configurations relative to regime-matched baseline. Top-left: paired speedup S (baseline at $S{=}1$). Top-right: mean latency reduction and throughput gain percentages vs baseline. Bottom-left: TTFT and TPOT percentage deltas vs baseline. Bottom-right: acceptance dynamics (α and B_{eff}) across the same configurations.

rises. This pattern indicates that fixed-policy drafting is vulnerable to acceptance/overhead imbalance and long-tail latency. Rather than presenting a separate unfinished contribution, we use this evidence to derive adaptive-policy design guidance: keep standard target-side verification while adapting control decisions online.

Advanced speculative variants such as EAGLE-3 and DFlash reduce drafting cost through new drafter architectures and dedicated training pipelines, often on very strong accelerators. Our objective is different: retain a same-family drafter/verifier stack deployable on consumer hardware, then recover speed and stability by adapting runtime policy (adaptive k, rescue switching, and tail fallback) instead of introducing a newly trained drafter.

A. Design objective

Let speculative-cycle latency be

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \tag{4}$$

where τ is expected accepted tokens per cycle. In fixed-policy speculative decoding, poorly matched samples can reduce τ and increase total verify steps, producing heavy-tail runtime. ACSD explicitly targets this effect by adapting draft policy at sample level.

B. Derived adaptive policy template

ACSD combines four online controls while preserving standard target-side verification:

1) **Adaptive k:** choose $k \in \{4, 8, 16\}$ from rolling acceptance, using larger blocks only when acceptance supports it. To avoid cold-start overshoot, each sample starts from base k before adaptive updates.
2) **Cascaded draft switch:** use 0.5B as a fast primary drafter and switch to 1.5B rescue drafting when acceptance remains low for consecutive verify steps, then enforce a cooldown before a new rescue episode to reduce oscillation.
3) **Tail guardrail:** if acceptance stays below a threshold after a minimum generated length for multiple consecutive checks, fall back to target-only autoregressive decoding for the remainder of that sample.
4) **Checkpointed execution:** persist partial CSV and verify-log artifacts at fixed intervals so interrupted runs retain recoverable progress.

This design preserves verifier compatibility and allows direct comparison with vanilla speculative decoding under the same manifests, prompts, metrics, and hardware.

C. How this guidance is used

Any future adaptive-policy experiment should be evaluated as a strict incremental study under the same protocol. We

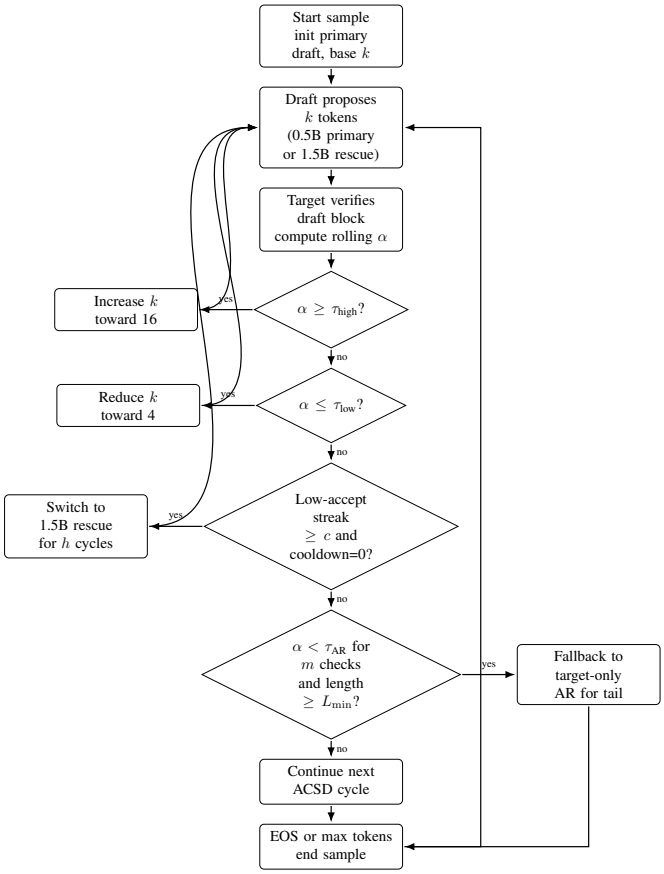

Fig. 3. Adaptive speculative-control template derived from fixed-policy results. The controller adapts block size from rolling acceptance, switches from a 0.5B primary to a 1.5B rescue drafter when acceptance remains low, applies rescue cooldown, and uses delayed target-only autoregressive fallback for pathological tails.

retain:

- the same target model and task manifests,
- the same deterministic and stochastic regimes,
- the same reporting metrics (S, α, B_{eff}, quality deltas),
- the same artifact schema for per-configuration CSV outputs, with ACSD metadata fields for adaptive-k usage, rescue usage, and fallback rate.

Primary comparison points are the current best fixed-policy speculative setting (k=4 deterministic) and the high-k settings (k=16) where fixed-policy decoding showed diminishing returns.

D. Scope in this paper

This section is intentionally scoped as design guidance derived from the fixed RTX4090 characterization in Section VI. Quantitative claims in this manuscript are restricted to the measured fixed-policy configurations.

VIII. Discussion

A. Main empirical takeaways

Three findings are consistent in the current run family. First, speculative decoding provides consistent acceleration across all tested configurations ($S > 1$ throughout). Second, speedup declines as k increases in this run family, with the strongest performance at k=4 and the weakest at k=16. Third, deterministic decoding outperforms stochastic decoding in 5 of 6 matched draft–k pairs.

Together, these results suggest that practical deployment should prioritise a smaller block size (k=4) and deterministic policy when latency is the primary objective.

From a positioning standpoint, this matters because demand for local and consumer-hardware LLM inference is increasing: the relevant question is no longer only "how fast can we decode in data centers," but "which decoding policy remains efficient and stable on constrained personal hardware."

We therefore present hardware-profiled operating points rather than universal defaults.

This framing is relevant to broader ICECCME-style engineering settings, including edge devices and local decision-support pipelines where predictable latency is a system-level requirement rather than only an NLP benchmark target.

B. Draft-size implications

The 0.5B draft produces the best single configuration (0.5B, k=4, deterministic), and also has a slightly higher mean speedup than 1.5B across the full grid in this run family. This indicates that draft-size selection is hardware- and budget-sensitive: the smaller draft can win on end-to-end speed even when the larger draft improves acceptance.

C. Speed versus quality

Quality drift is task-dependent. CNN/DailyMail and MMLU remain relatively stable across settings, while GSM8K is more variable, especially under stochastic decoding. A conservative filter ($\max|\Delta| \leq 1$ across all three tasks) retains a small subset of configurations, among which 1.5B, k=8, deterministic remains best. This supports a practical policy: use deterministic mode with k=4 for production-like latency targets, and treat stochastic settings as quality-sensitive configurations requiring additional calibration. Final winner claims should be interpreted together with paired confidence intervals and corrected pairwise tests from a coherent artifact family.

IX. Threats to validity

Artifact consistency across reruns. Some baseline files were regenerated after initial summary artifacts were produced, which can create mismatches between raw baseline totals and summary-implied baselines. To mitigate this, we use one coherent summary source for comparative claims and report this limitation explicitly.

Single-hardware scope. All results are from one RTX 4090 Laptop. Relative rankings are informative for this deployment class, but absolute speedups may shift on other GPU generations. In this run family, CUDA-graph metadata indicates pending/zero capture rate, so reported gains reflect the same code path under fallback execution rather than successful graph replay.

Model-family scope. All reported runs use same-family pairings (Qwen2.5 target and drafts). Cross-family pairings

may therefore exhibit materially different acceptance and quality behavior.

Evaluation breadth. The benchmark set spans math reasoning, knowledge QA, and summarisation, but does not exhaust safety, factuality, or long-context behaviour. ROUGE-L in particular is an incomplete summarisation quality proxy.

X. Conclusion

This paper provides a controlled consumer-GPU evaluation of speculative decoding with a Qwen2.5-3B target and same-family drafts. The current evidence shows consistent acceleration across all tested configurations, with best observed performance at 0.5B, k=4, deterministic (S=3.0674). We find that increasing k from 4 to 8 and 16 reduces net speedup in this setup, and that deterministic decoding is generally faster than stochastic decoding for matched settings.

These findings yield an actionable operating recommendation for the present hardware budget: prefer deterministic speculative decoding at k=4, with 0.5B draft as the default fast path and larger drafts reserved for quality-sensitive scenarios. Future work will use the adaptive-strategy guidance in Section VII under the same manifests and metrics to test whether adaptive draft length and cascaded draft-model switching can improve tail latency without sacrificing quality control. The quantitative claims in this manuscript remain restricted to the measured fixed-policy configurations.

Acknowledgements

The authors gratefully acknowledge the Indonesian Endowment Fund for Education Agency (Lembaga Pengelola Dana Pendidikan - LPDP) for supporting this research through a scholarship awarded to Samsu Sempena (Grant ID: LOG-21706/LPDP.3/2024).

References

[1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," in *arXiv preprint arXiv:2302.01318*, 2023.

[3] M. Stern, N. Shazeer, and J. Uszkoreit, "Blockwise parallel decoding for deep autoregressive models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," *arXiv preprint arXiv:2401.10774*, 2024.

[5] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," *arXiv preprint arXiv:2401.15077*, 2024.

[6] ——, "EAGLE-2: Faster inference of language models with dynamic draft trees," *arXiv preprint arXiv:2406.16858*, 2024.

[7] ——, "EAGLE-3: Scaling up inference acceleration of large language models via training-time test," *arXiv preprint arXiv:2503.01840*, 2025.

[8] G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang, "DiffuSpec: Unlocking diffusion language models for speculative decoding," *arXiv preprint arXiv:2510.02358v1*, 2025, https://arxiv.org/abs/2510.02358v1.

[9] J. Sandler, J. K. Christopher, T. Hartvigsen, and F. Fioretto, "SpecDiff-2: Scaling diffusion drafter alignment for faster speculative decoding," *arXiv preprint arXiv:2511.00606*, 2025.

[10] Z. An, H. Bai, Z. Liu, D. Li, and E. Barsoum, "PARD: Accelerating llm inference with low-cost parallel draft model adaptation," *arXiv preprint arXiv:2504.18583*, 2025.

[11] J. Chen, Y. Liang, and Z. Liu, "DFlash: Block diffusion for flash speculative decoding," *arXiv preprint arXiv:2602.06036*, 2026.

[12] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[13] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[14] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," *arXiv preprint arXiv:2009.03300*, 2020.

[15] S. Narayan, S. B. Cohen, and M. Lapata, "Don't give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.